Welcome to the Ultimate DSA 2025–2026 Resource Sheet

"Everyone starts confused. The only difference is—some stay there, others learn how to think."

Hey Everyone,
If you've ever felt lost while learning Data Structures and Algorithms (DSA) — you're not alone. Almost every developer, including me, once struggled with where to start, what to learn, and how to actually build logic.

This sheet is here to fix that.

You'll find:
✅ A clear, structured path to learn DSA in 3 months
✅ The best free resources for every topic
✅ A list of problem patterns to master interviews
✅ Practical tips, logic-building techniques, and platform links
✅ All organized in the same order as explained in the video

⚡ Whether you're a beginner, a college student, or someone restarting your coding journey — this roadmap will help you learn DSA smartly, not blindly.

So take it slow, follow it step by step, and remember —
🧠 DSA is not about intelligence. It's about patience, consistency, and pattern recognition.

All the best - With ❤️ **from HarshaVerse**
Let's get started 💪.

Part 1: Do You Still Need DSA in 2025/2026?

✅ Why DSA Still Matters

- DSA builds **logic, clarity, and structured thinking**.
- It's the **gym for your brain** — hurts, but keeps you sharp.
- Big tech (FAANG / MAANG) → still heavy on DSA.
- Startups → prefer projects & portfolios.

HarshaVerse

Part 2: Common Beginner Mistakes

🚫 3 Classic Mistakes

1. **Copy-Paste Learning** — reading solutions without trying.
2. **Quantity over Quality** — solving 500 problems without learning patterns.
3. **Skipping Basics** — directly jumping to DP/Graphs.

Quote: *"Don't try to run a marathon before learning how to walk."*

Common Mistakes By Beginners	Solutions/Hacks
Copy-Paste Learning — reading solutions without trying.	Read this to a better Problem Solver
Quantity over Quality — solving 500 problems without learning patterns. (More about this in Logic Building part)	LeetCode was HARD until I Learned these 15 Patterns
Skipping Basics — directly jumping to DP/Graphs.	Follow the order mentioned below in DSA prerequisites and 90 days DSA roadmap.

Part 3: Logic Building & Thinking Like a Coder

3.1 🧩 Conceptual Understanding

- Start with **Easy Problems**.
- Map every DSA concept to a **real-life example**.
    - Queue → Cinema line (FIFO)

- Stack → Plate pile (LIFO)
- Graph → Cities & roads (Nodes + Edges)
- Dijkstra's → Finding shortest route with weights (traffic, distance, etc.)

Check out this article: 9 Hacks to Improve Logic Building Skills

3.2 Coding Patterns (Mastering the Logic Behind Problems)

"Stop solving 1000 problems. Start mastering 16 patterns."

"Don't memorize problems. Recognize **patterns**."

This will be the single most important thing in this whole journey. The earlier you realise this the better the 90 days journey would be.

Why This Matters

Most coding interview problems are just variations of a few repeatable logic patterns.
Once you identify and internalize these, you'll solve any problem faster — not by memorizing, but by recognizing.
This is the foundation of real problem-solving.

How to Approach

1. **Go in order** — Beginner → Intermediate → Advanced.
2. For every pattern, first **read** or **watch** to understand the logic, then **solve 3–5 problems**.
3. Maintain a small *Pattern Tracker* (Pattern name | Example problems | Key insight).
4. Revisit old patterns regularly — repetition cements recall.

Harsha verse

🟢 **Preference 1**

Resource Type	Name	Link	Notes
GitHub	Several Coding Patterns by Chanda Abdul	GitHub Link	Clear explanations and examples of common interview patterns.
GitHub	Awesome Leetcode Resources by AshishPS	GitHub Link	Beginner-friendly list of problem sets and guides.
YouTube	“15 Most Common Coding Patterns”	YouTube Link	Great visual introduction to common DSA problem-solving patterns.

🟡 **Preference 2**

Resource Type	Name	Link	Notes
GitHub	DSA Patterns Roadmap by Kushal Vijay	GitHub Link	Structured roadmap with categorized problem types.
GitHub	MAANG Interview Prep 100 DSA by VikashPR	GitHub Link	Covers pattern-based problems often asked in top company interviews.
YouTube	“8 Patterns to Solve 80% of Leetcode Problems”	YouTube Link	Focuses on core templates that reappear in most coding problems.

🔴 **Preference 3**

Resource Type	Name	Link	Notes
GitHub	Java DSA Repository by Debapriyo007	GitHub Link	Java-based in-depth implementations of advanced patterns.
GitHub Gist	Pattern Summary by MathCodes	GitHub Link	Concise summary of advanced problem-solving templates.
YouTube	“Advanced Coding Templates and Patterns”	YouTube Link	Explains high-level templates like sliding window, backtracking, graphs, and DP.

🧩 Suggested Learning Order

1. Start with **Beginner GitHub + 15 Patterns video** to build familiarity.
2. Practice 3–5 problems for each pattern (e.g. Sliding Window, Two Pointers, BFS/DFS).
3. Move to **Intermediate** — focus on real interview-style problems and pattern variations.
4. Finish with **Advanced** — deep dive into dynamic programming, graph algorithms, and complex recursion.
5. Revisit and summarize patterns in your *Pattern Tracker*.

Perfect — here's how to make your **3.3 — Brute Force → Optimized Thinking** section match the clarity and insight of your previous `3.2` section.

I've added **one-liners for context**, a clear **approach framework**, and a **handpicked set of the best YouTube + GitHub resources** that teach *optimization thinking* (not just syntax).
No fluff — just high-value, trackable learning.

3.3 Brute Force → Optimized Thinking

> *"Think slow first, then optimize fast."*

🎯 Why This Matters

Most students jump straight to optimized code and miss the logic behind it.
But the best problem solvers **first think brute-force**, understand every inefficiency, and **then refine logically**.
That's what builds real *algorithmic intuition* — the ability to optimize under pressure.

🧭 Step-by-Step Approach

1. **Start with a brute-force solution** — don't fear $O(n^2)$. Clarity first.
2. **Write down what's repeating** — loops, recalculations, unnecessary traversals.

Harsha verse

3. **Ask "what if I could store or skip this?"** → that's where data structures and patterns help.

4. **Apply optimization** using known approaches:

    - Hashing / Maps for lookups
    - Sorting + Two Pointers
    - Sliding Window for subarrays
    - Prefix/Suffix logic for accumulative patterns
    - Binary Search on answers

5. **Compare time complexities** — see the evolution from $O(n^2) \rightarrow O(n \log n) \rightarrow O(n)$.

Best Resources to Build Optimization Mindset

Type	Name	Link	Why It's Great
📘 Article	**Brute Force to Optimal Solutions – GeeksforGeeks**	Read Here	Covers the philosophy + examples of transforming brute-force logic.

🧩 Suggested Practice Flow

1. Pick a random LeetCode Easy/Medium problem.
2. Solve with **brute force** — even if inefficient.
3. Identify repeating computation or unnecessary loops.

Harsha verse

4. Ask which **pattern or data structure** could remove that repetition.
5. Rewrite optimized version — note down *what changed* and *why it worked*.
6. Repeat for 3–5 problems each week — pattern recognition will become instinctive.

Part 4: DSA Prerequisites — *Before Starting the 90-Day Plan*

⚠️ If you don't have these basics, the 90-day plan becomes much harder — take **10-15 days** to finish these first.

Category	What to Learn	Why It Matters	Quick Starter Links
🧑‍💻 **Programming Fundamentals**	Pick one language (Python / C++ / Java). Learn syntax, loops, conditionals, functions, I/O. Understand arrays, strings, maps/dictionaries, sets. Get used to standard libraries (STL / Collections / Python built-ins).	You'll write all your DSA logic in one consistent language.	- Python for Beginners – FreeCodeCamp - C++ STL Cheatsheet – GFG
⏱️ **Complexity & Problem Reading**	Learn Big-O notation (O(1), O(log n), O(n), O(n log n), O(n^2)). Understand constraints & time limits.	Helps you predict feasible solutions and avoid TLEs.	- Big O Notation Explained – CS Dojo - Big-O for Humans -Big O notation Cheat Sheet

Basic Coding Warm-up (Before DSA)

Start with 15–20 simple problems to strengthen logic, loops, and pattern thinking.
This helps you warm up before you jump into real DSA problems.

Harsha verse

Topic	Description	Practice Links
🎨 **Pattern Problems**	Build logic for loops, conditionals, and nested structures.	Java Pattern Programs – GFG
💡 **Basic Programming Problems**	Simple problems on numbers, strings, arrays — perfect warm-up before arrays & recursion.	Basic Programming Problems – GFG
💻 **Top 100 Coding Questions**	Curated beginner-friendly coding problems across multiple languages.	Top 100 Codes – PrepInsta

Part 5: 90 Days DSA roadmap

You've done the hard part — built your base, understood how to think, and learned the patterns that power 90% of DSA problems.
Now it's time to **apply everything you've learned** — step by step, with structure and purpose.

These next 90 days aren't just about solving problems —
they're about **training your brain to think like an engineer.**

You'll start with simple problems and slowly move toward complex ones,
seeing how logic, optimization, and patterns come together.

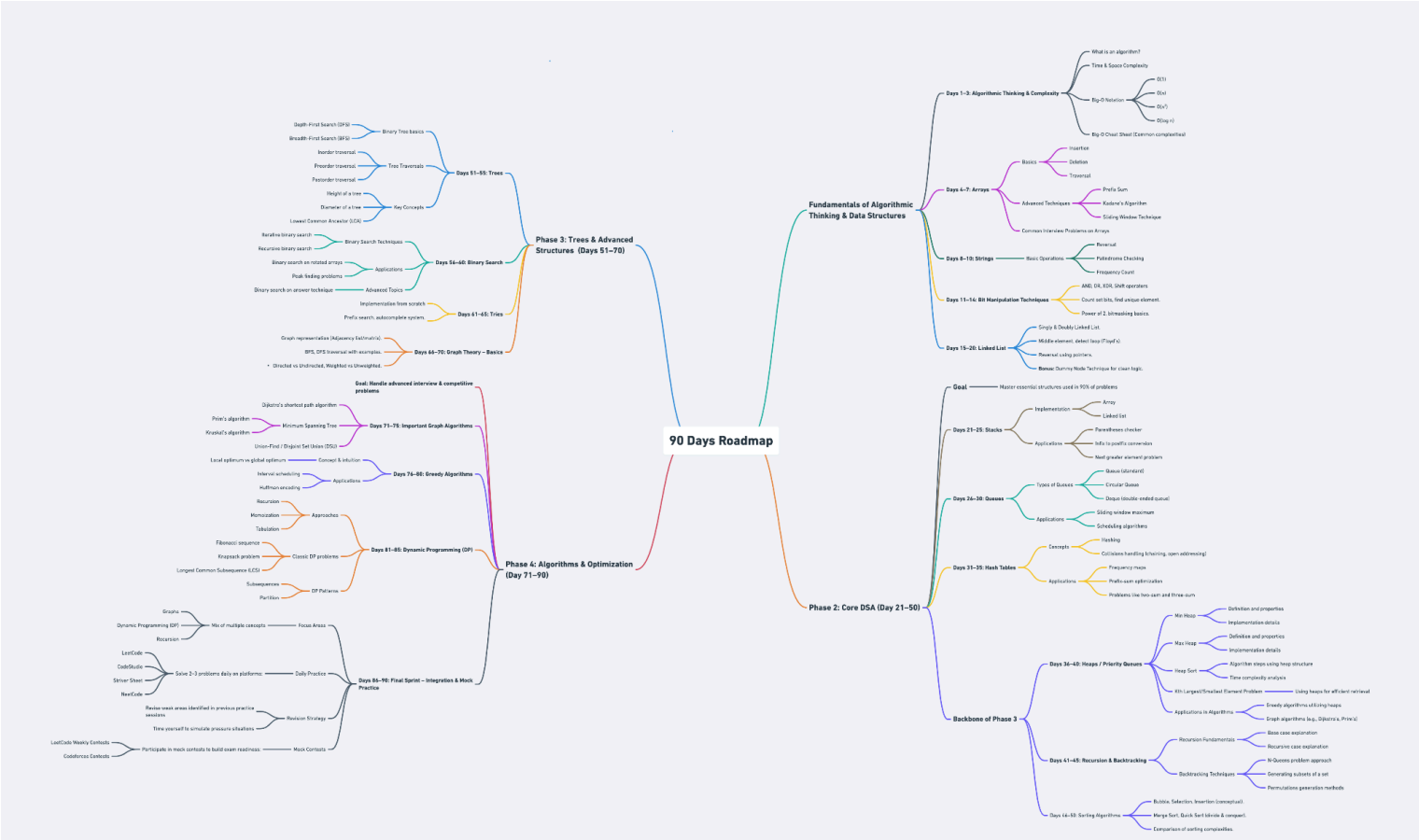
📘 **Checklist Before You Continue**

- ✅ You've gone through the prerequisites section or placement sheet
- ✅ You've practiced basic coding & pattern problems
- ✅ You understand brute-force vs optimized thinking
- ✅ You're ready to commit to 90 consistent days

This is where the **real journey begins.**
Take it one day at a time — and by the end, you'll not only solve problems…
you'll *understand how to think in DSA.*

Harsha verse

Also You can find detailed resources on getting started with each of the programming languages and DSA Questions topic wise along with multiple options in my placement sheet.
[Harsha Verse Placement sheet]

And Follow the Below order phase wise to finish all the topics in 90 days.

Click on the MindMap to view this clearly.